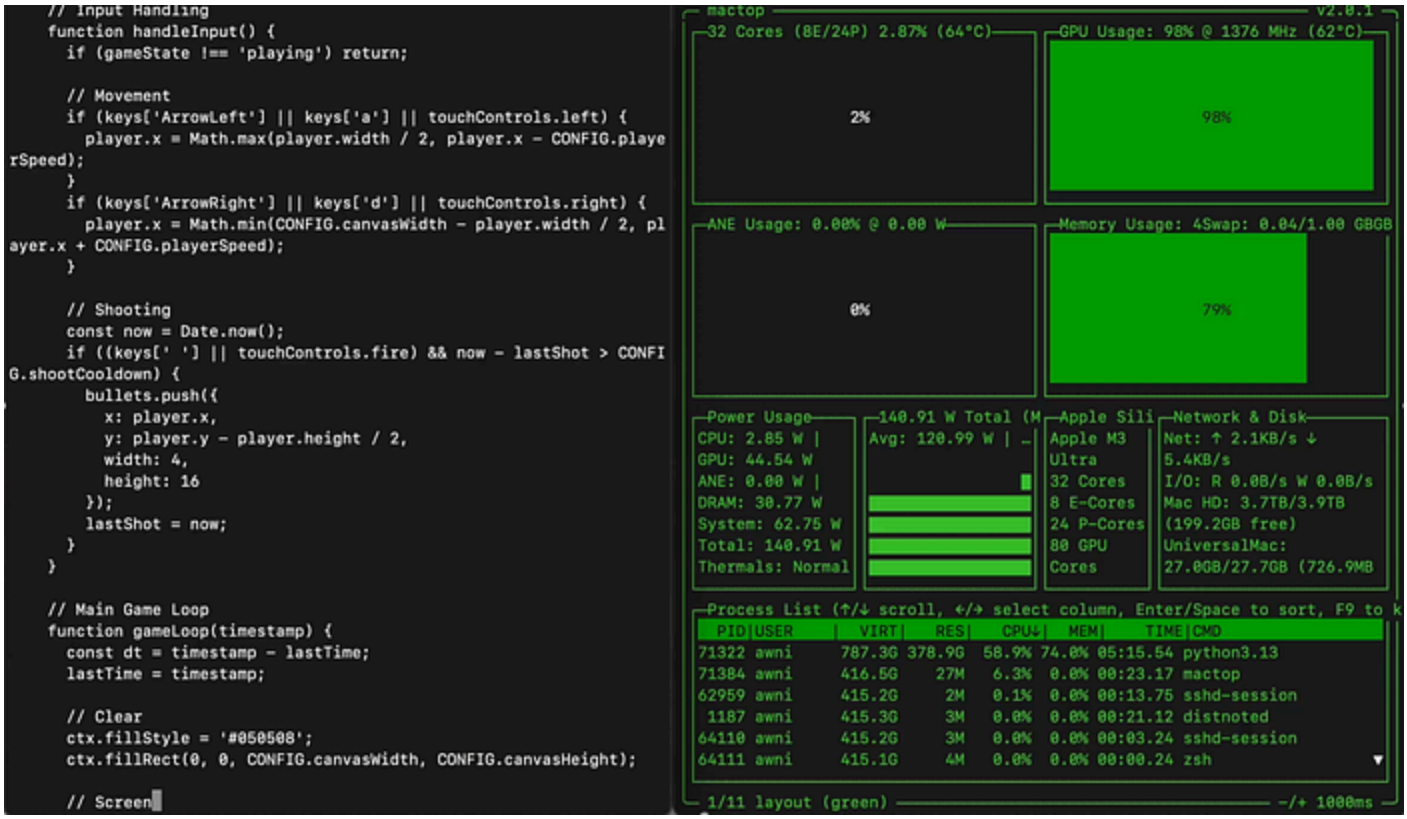
Milestone: 20GB Reached

We've reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original



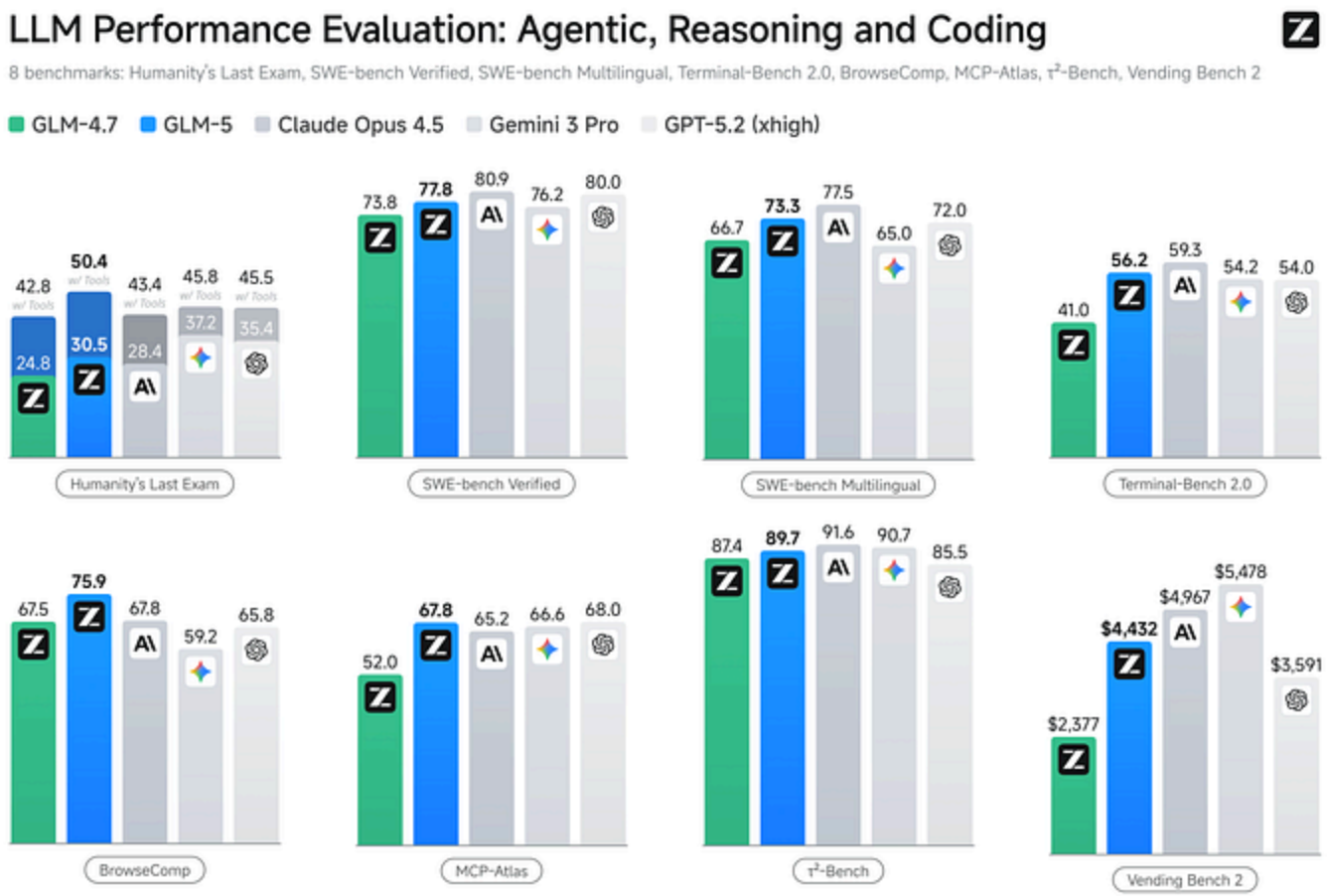
GLM-5 joins Opus and Codex: Long-horizon Agentic Tasks

One of the most interesting parts of the GLM-5 launch is that you can run an open-weights model inside a proprietary agentic coding...

Agent Native Follow

~8 min read · February 12, 2026 (Updated: February 12, 2026) · Free: No

One of the most interesting parts of the GLM-5 launch is that **you can run an open-weights model inside a proprietary agentic coding workflow** and get something close to frontier-grade day-to-day usefulness **without paying frontier-grade subscription prices.**



GLM-5 works inside Claude Code, with comparable performance at fraction of the cost:

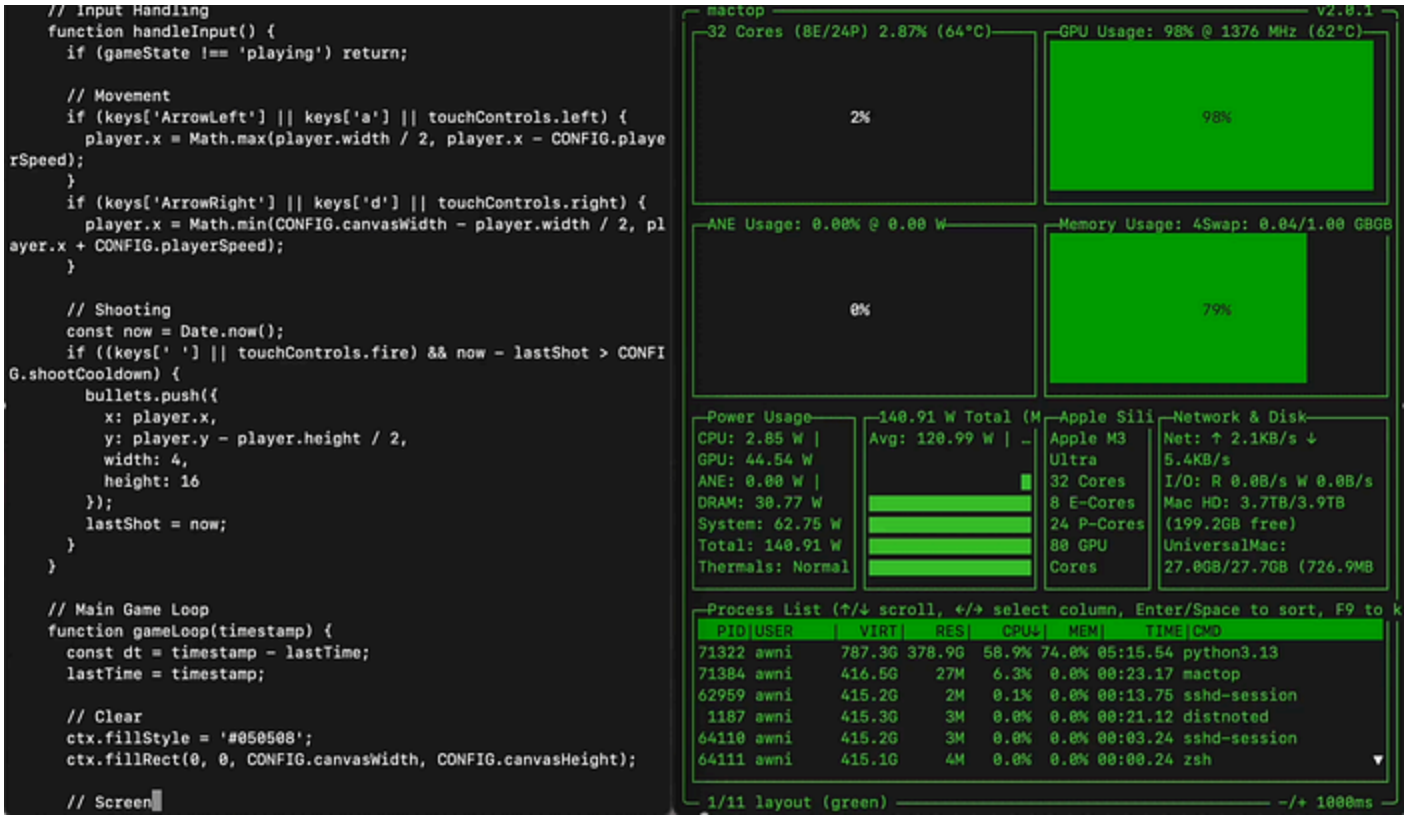
- Claude Opus 4.5: $5/$25
- GPT 5.2 Codex: $1.75/$14
- GLM-5: $0.80/$2.56

GLM-5 is 6x cheaper than Opus on input and 10x cheaper on output.

It also has the lowest hallucination rate among LLMs tested by ArtificialAnalysis.

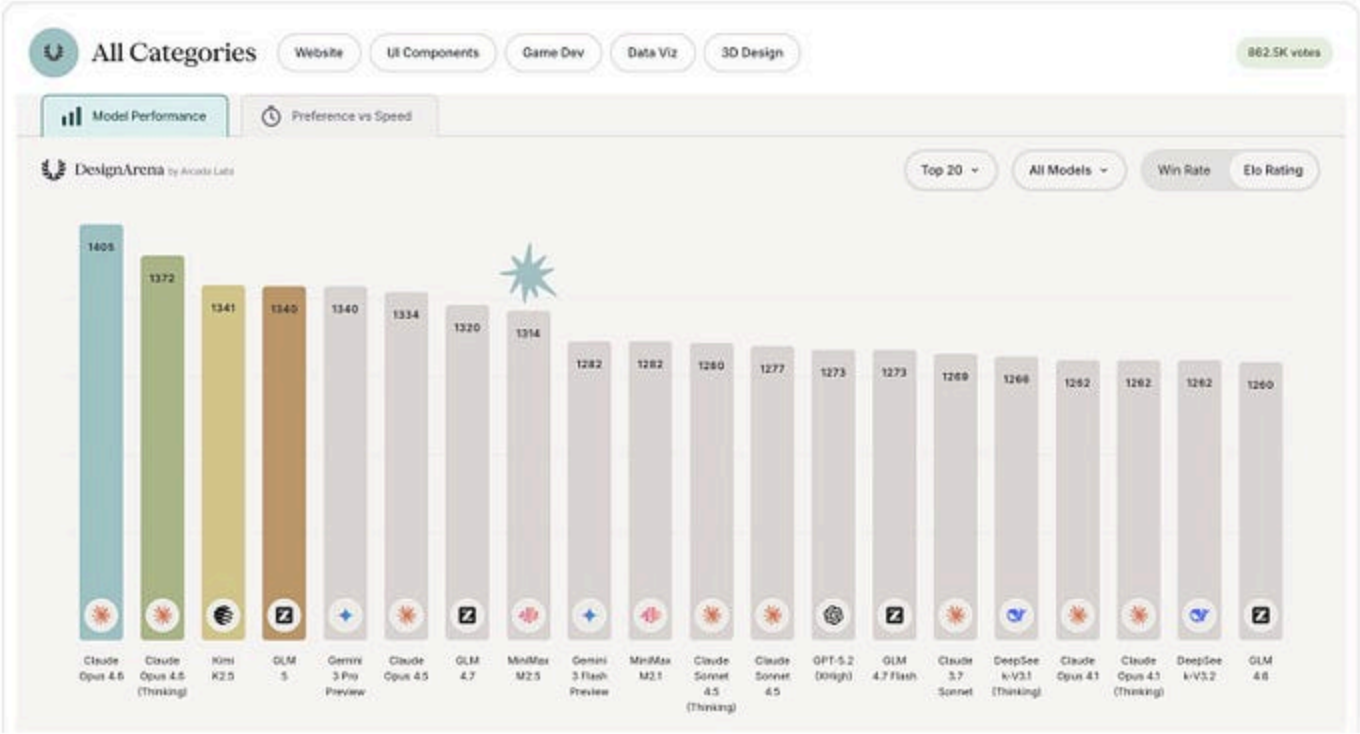
It has already been quantized to MLX, you can run it with mlx-lm on a single 512GB M3 Ultra in Q4.

> It generated a highly functional space invaders game using 7.1k tokens at 15.4 tok/s and 419GB memory.



As Awni Hannun reported on X

It also ranks **4th on DesignArena**, a design-focused benchmark.



I'll pull out the most decision-relevant details from the launch and the developer chatter, because this is where people get misled by benchmark charts and miss the operational story.

> I'm writing a deep-dive ebook on **Agentic SaaS,** the emerging design patterns that are quietly powering the most innovative startups of 2026.
>
> **You can grab the first chapter here:** [Agentic SaaS Patterns Winning in 2026](), packed with real-world examples, architectures, and workflows you won't find anywhere else.

Six non-obvious takeaways

(1) "Open weights" can show up as "a new model option in your existing tools."

GLM-5 is released with weights under the MIT License on Hugging Face and ModelScope, but it's also offered via hosted APIs and positioned to work in existing coding agent workflows (including Claude Code and OpenClaw).

(2) It's huge and it's built to be served, not "run on your laptop." GLM-5 has 744B total parameters with 40B active (MoE), plus long-context optimization via DeepSeek Sparse Attention (DSA).

It still expects serious serving infrastructure.

(3) Tool use isn't a side quest In the published benchmark table, GLM-5's Humanity's Last Exam score jumps meaningfully when "with tools" is enabled (and that pattern shows up across models).

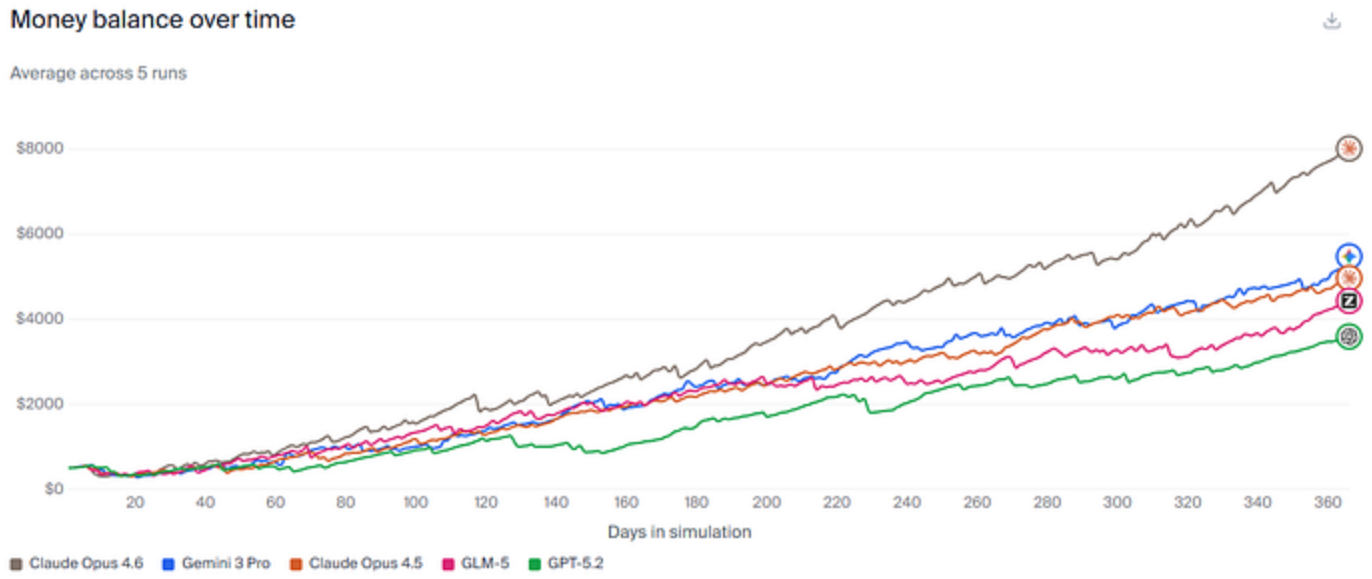
If you're building agents, you should read that as: your tools layer is part of your model choice.

(4) The training infrastructure ("slime") changes the cadence of improvements.

They explicitly call out RL inefficiency at LLM scale, and "slime" as an asynchronous RL infrastructure to improve throughput and allow finer post-training iteration. That's a signal about how fast the model family may evolve, and what kinds of agent behaviors they're optimizing for.

(5) Long-horizon agents are being benchmarked like businesses now.

Vending-Bench 2 is a one-year simulated vending machine business. GLM-5 lands near the top of the leaderboard among widely discussed models. If you ship agents that need to stay coherent over long trajectories, this genre of eval is closer to your reality than most "single-turn" tests.



(6) There are still "open model" pain patterns: coherence, stability, and multimodality gaps.

There are obviously expected occasional global coherence issues, higher variance ("half the time you get something weird"), and limitations like text-only input for GLM-5 (no native image understanding).

That doesn't make the model bad but you need guardrails and fallbacks.

The main story is stack vs stack:

- Model weights (control, licensing)
- Serving layer (latency, cost, scaling)
- Agent framework + tools (execution, safety, UX)
- Evaluation harness (variance, reliability)

You can now mix these layers in ways that were awkward a year ago.

Open-weights vs proprietary

Let's define terms quickly, because people talk past each other:

- **Open-weights**: you can download model weights and run them (subject to hardware + tooling), and often host them yourself. **Open** here usually refers to weights availability and license, not necessarily training data or training recipe transparency.
- **Proprietary or closed-weight**: you consume the model via an API or a hosted product; weights are not released.

How do you decide?

Dimension	Open-weights tends to win when...	Proprietary tends to win when...	What people forget
Cost at scale	You can amortize infra and control token usage patterns	You're early / spiky / don't want to run GPUs	"Cheap per token" isn't cheap if your agent loops
Control & portability	You need on-prem, custom deployment, or long-term leverage	You prioritize speed of integration	Toolchain lock-in can happen even with "open" models
Latency & reliability	You can tune serving, batch, cache, and accept tradeoffs	You need consistent low-variance UX now	Variance kills product trust more than raw quality
Capabilities	Your use case is mostly text/coding/agentic planning	You need best-in-class multimodal or niche strengths	"Best benchmark model" can still fail your workflow
Compliance & data	You need tight control over data handling	You're okay with vendor compliance posture	The risk is often in tools + logs, not just prompts
Iteration speed	You can swap models, fine-tune, or customize	You want vendor to improve for you	Your evaluation harness is what actually accelerates you

If you can answer these, you're already ahead of most developers:

1. Is your workload **interactive** (humans waiting) or **batchy** (agents grinding)?
2. What's your **failure mode**: wrong answer, unsafe action, or endless wandering?
3. Do you need **native multimodality** (images) or can tools cover it?
4. Will you run this in a **coding agent** (long contexts, tool calls), or plain chat?
5. How much **variance** can your UX tolerate?
6. What's your **switching cost** if you regret the choice in 90 days?

Keep those answers visible while you read benchmark leaderboards.

What's real, what's risky, and when not to use it

GLM-5 bundles together a few themes that are worth understanding as patterns.

Sparse attention for long context (DSA)

DeepSeek Sparse Attention (DSA) is positioned as a way to reduce deployment cost while preserving long-context capacity. Mechanistically, sparse attention is about not paying full price for every token attending to every other token.

Asynchronous RL infrastructure (slime)

Reinforcement learning can bridge competence to excellence, but scaling RL for LLMs is inefficient. Their answer was **slime**, an asynchronous RL infrastructure to improve throughput and enable finer post-training iteration.

Models moving from chat to work

Foundation models are moving from "chat" to "work", team is explicitly calling out end-to-end document generation, `.docx`, `.pdf`, `.xlsx`, as a first-class outcome.

Playbook for picking your model stack

This is the part I wish someone had handed me before I wasted weeks debating "best model" instead of building a harness.

Step 1: Choose your stack archetype, not your winner

Pick one of these intentionally:

Archetype A: Proprietary flagship as the default

- Great when you need the strongest baseline, lowest variance, and a single throat to choke.
- However costs climb quietly and you optimize late.

Archetype B: Open-weights as the workhorse + proprietary for edge cases

This is the pattern implied by the GLM-5. You can use an open-weights model where it's "good enough" for long grinding work, and reserve premium models for high-stakes or multimodal tasks.

Archetype C: Multi-model routing based on task type and risk

- Fast, powerful, and brittle unless you have routing logic + evals.
- Worth it when you already operate at scale and model costs are material.

Step 2: Run a one-week eval that measures variance, not vibes

Here's a lightweight plan:

Day 1–2: Build a task set from your own repo / tickets

- 20–50 tasks max.
- Include boring tasks (refactors, test fixes) and "dangerous" tasks (auth, payments, migrations).

Day 3–4: Measure

- Pass rate (did it work?)
- Time-to-first-useful-output
- Tool call failures
- "Went weird" rate (nonsense, incoherent plan, broken assumptions)

You can explicitly flag stability/variance as a differentiator you should test, not assume.

Day 5: Add a long-horizon test Even one synthetic long-run task matters.

Can it maintain state, avoid thrashing, and converge?

Benchmarks like Vending-Bench 2 exist because this failure mode is real.

Step 3: Make switching cheap on purpose

If you do nothing else, do this:

- Wrap model calls behind a single interface in your app.

- Version your prompts.
- Log tool traces.
- Keep a rollback path that doesn't require heroics.

Because today it's "remove two env vars." Tomorrow it's "untangle three months of prompt-model co-adaptation."

Where I land (slightly opinionated), and questions for you

My take is **open-weights vs proprietary is no longer a purity debate.**

It's an engineering decision about where you want control and where you want leverage.

GLM-5 is a compelling example of the new middle ground:

- Open weights under MIT on major model hubs.
- Pushing long-horizon agent evaluation into "runs a business for a year" territory.
- Explicit investment in RL infrastructure (slime) to tighten post-training iteration loops.
- And maybe most importantly, **showing up inside the tools developers already use**, which changes adoption dynamics more than any single benchmark number.

But I'm not going to pretend the risks disappear:

- If your UX can't tolerate variance, you'll still pay for a flagship.
- If you need native vision in your core loop, text-only models force tool workarounds.
- If you don't invest in evals and rollback paths, you will get surprised in production.

Questions I'd genuinely love to hear your answers to

1. What's your real bottleneck right now: **model quality, tool reliability, or evaluation discipline**?
2. If you've adopted open-weights in production, where did it break first: **latency, variance, or ops burden**?
3. For agentic workflows: do you optimize for **best single model**, or **best stack** (router + tools + fallback)?
4. What's your "switching cost" today — could you roll back a model choice in under an hour?

If you reply with your use case (agent vs chat, text vs multimodal, interactive vs batch), I'll tell you which archetype I'd pick and what I'd measure in week one.

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

Deep Research with 96,000+ Trajectories - Completely Offline

Deep Research with 96,000+ Trajectories - Completely Offline Imagine synthesizing human-like research trajectories...

medium.com

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic...

medium.com

Qwen3-Coder-Next Failed in the Scariest Way: 10 Practical Tests

Benchmarks are the LinkedIn of LLMs. Every model looks unstoppable.

medium.com

Clawdbot Lite: 99% Smaller, 4000 Lines, Same Core Power

People are building Polymarket bots, multi-agent product teams, crypto traders, coding agents, business operators with...

medium.com

Vectorless RAG for Agents: PageIndex Is Why Their Demo Works and Yours Needs "Context"

When I'm building a RAG apps, the thing that quietly kills quality (and budgets) is turning a document into a pile of...

medium.com

Fully Local Agentic Coding on localhost: Claude Code, Codex, llama.cpp, Ollama and Docker MR

This is the setup I wish I had the first time I tried to run Claude Code and Codex-style workflows against local...

medium.com

Open Source Perplexity: Perplexica is Self-Hosted AI Search With Citations

Perplexica is an open-source, AI-powered search engine inspired by Perplexity-style UX.

medium.com
#glm-5#openai-codex#claude-opus#agentic-ai#agentic-coding

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel